CURSO PARA INTEGRADORES DO HUBSAÚDE · MÓDULO 4 DE 6

4 Autenticar

Antes de aceitar um registro, o HubSaúde precisa saber quem o está enviando. Neste módulo, você vê como o seu sistema comprova a identidade com o certificado digital e recebe em troca o token de acesso. Primeiro, você executa cada passo pelo terminal, para ver o que acontece por dentro; depois, obtém o mesmo resultado com a biblioteca oficial, em Java.

AO FINAL, VOCÊ SERÁ CAPAZ DE

- ✓ explicar por que o HubSaúde não usa usuário e senha;
- ✓ descrever o conteúdo de um `client_assertion` e a sua função;
- ✓ obter um token de acesso no Simulador, pelo terminal e pela biblioteca oficial;
- ✓ explicar o vínculo entre o token e o certificado do sistema;
- ✓ diagnosticar as falhas de autenticação mais comuns.

COMO O MÓDULO ESTÁ ORGANIZADO

1. **Conceitos:** seções 4.1 a 4.5
2. **Prática guiada:** 6 passos no terminal
3. **Verificação:** perguntas, glossário e respostas

VOCÊ VAI PRECISAR

- O Simulador em funcionamento e o sistema `meu-sistema` cadastrado, como no módulo 2.
- Na pasta `curso-hubsaude`, os arquivos `client-key.pem`, `client-cert.pem` e `simulador-server.pem`.
- Para o passo 6: o Java 21, o Maven 3.9 ou mais recente e o Git.

ASSISTA AOS VÍDEOS

Aponte a câmera do celular para o código.

SEÇÕES 4.1 A 4.5 E PASSO 6

Como o sistema obtém o token de acesso

5 min 47 s, com narração e legendas

DÚVIDAS E SUGESTÕES

Se algum passo ficar difícil, escreva para o suporte do HubSaúde, em `suporte@saude.go.gov.br`, com o assunto "Curso para integradores, módulo 4". A versão mais recente deste módulo, com os vídeos, está em `https://sesgo-ti.github.io/hubsaude/curso/autenticar/`.

Copiar de um PDF pode alterar aspas, hífens e quebras de linha. Para os comandos e arquivos, prefira a versão do módulo no portal ou os arquivos para baixar, em `https://sesgo-ti.github.io/hubsaude/curso/materiais/`.

PARTE 1

Conceitos

4.1 Identidade sem senha

Muitos serviços identificam quem os acessa por usuário e senha ou por uma chave de API. Os dois modelos têm o mesmo ponto fraco: o segredo precisa viajar até o servidor a cada acesso e, se for copiado, quem o copiou passa a se passar pelo sistema.

O HubSaúde não usa nenhum dos dois. O sistema comprova a identidade com o **certificado digital**, apresentado no módulo 2: a chave privada assina os pedidos e nunca sai do seu sistema, e a parte pública, conhecida pelo servidor desde o cadastro, confere essas assinaturas.

Analogia

Pense em uma assinatura reconhecida em cartório. Só o dono assina, mas qualquer um confere a assinatura pelo registro do cartório. A chave privada faz o papel da mão que assina; o certificado, o do registro.

Consequência

O servidor nunca recebe o segredo. Ele recebe apenas pedidos assinados, que não servem para produzir novas assinaturas.

Em troca da prova de identidade, o sistema recebe um **token de acesso**: uma autorização temporária que acompanha cada envio. O processo segue o padrão internacional SMART Backend Services, construído sobre o OAuth 2.0.

4.2 A conexão mTLS

Quando você acessa um site seguro, só o site apresenta um certificado: o navegador confere que está falando com o site certo, mas o site não sabe quem você é. No **mTLS**, a letra m vem de *mútuo*: os dois lados apresentam um certificado.

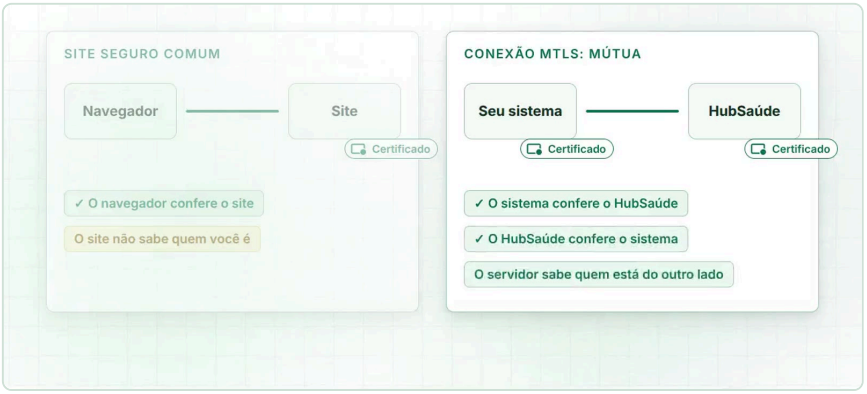


Figura 4.1. Em um site comum, só o servidor se identifica. Na conexão mTLS, o sistema e o HubSaúde conferem o certificado um do outro. Quadro do vídeo "Como o sistema obtém o token de acesso".

O seu sistema confere o certificado do HubSaúde, e o HubSaúde confere o certificado do seu sistema. Assim, antes mesmo de ler o pedido, o servidor já sabe quem está do outro lado. É por isso que o arquivo `simulador-server.pem`, baixado no módulo 2, aparece em todos os comandos deste módulo: com ele, o seu lado confere o Simulador.

4.3 Os três passos para obter o token

1. **Descoberta**

   O sistema consulta o endereço `/.well-known/smart-configuration` e obtém o endereço do servidor de autorização, como visto no módulo 1.

2. **Prova de identidade**

   O sistema monta um pequeno documento, o `client_assertion`, e o assina com a chave privada.

3. **Troca pelo token**

   O sistema envia o `client_assertion` pela conexão mTLS. O servidor confere a assinatura e o certificado da conexão e devolve o token.

> **OBSERVE**
> No terceiro passo, o servidor confere duas coisas diferentes: se a assinatura do documento é do sistema cadastrado e se o certificado apresentado na conexão é o mesmo do cadastro. As duas precisam conferir.

4.4 O client_assertion

O `client_assertion` funciona como um bilhete assinado: diz quem pede, para qual servidor, e vale por poucos minutos. Tecnicamente, é um **JWT**, um texto em três partes separadas por ponto: o cabeçalho, o conteúdo e a assinatura.



Figura 4.2. As três partes do `client_assertion` e os campos do conteúdo. Quadro do vídeo "Como o sistema obtém o token de acesso".

Campo	Significado	Por que existe
`iss` e `sub`	O `client_id` do sistema	Diz quem pede.
`aud`	O endereço do servidor de autorização	Impede que o bilhete seja aproveitado em outro servidor.
`iat` e `exp`	O momento da criação e o fim da validade, poucos minutos depois	Limita o tempo em que um bilhete interceptado teria valor.
`jti`	Um identificador único	O mesmo bilhete não pode ser usado duas vezes.

> **DICA**
> As duas primeiras partes de um JWT são apenas codificadas, e não criptografadas: qualquer um consegue lê-las, como você fará no passo 2. O que impede a falsificação é a terceira parte, a assinatura.

4.5 O token: validade, escopos e vínculo

Validade e escopos

O token vale por **cinco minutos** e traz os escopos pedidos, limitados ao que foi liberado no cadastro. Um escopo não liberado resulta no erro `invalid_scope`.

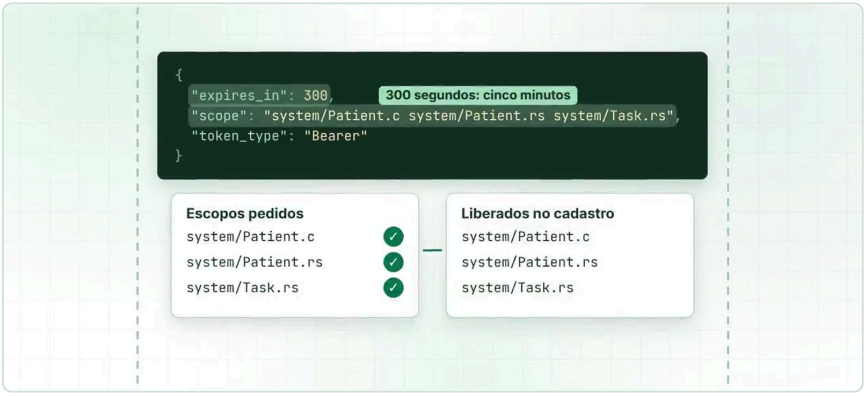


Figura 4.3. A resposta do servidor: a validade em segundos e os escopos concedidos, conferidos com o cadastro. Quadro do vídeo "Como o sistema obtém o token de acesso".

Vínculo com o certificado

O token fica vinculado ao certificado apresentado na conexão, por meio do campo `cnf`, que guarda a impressão digital do certificado. Toda chamada seguinte precisa apresentar o **mesmo certificado**. Assim, um token interceptado não serve a quem não possui a chave privada correspondente.

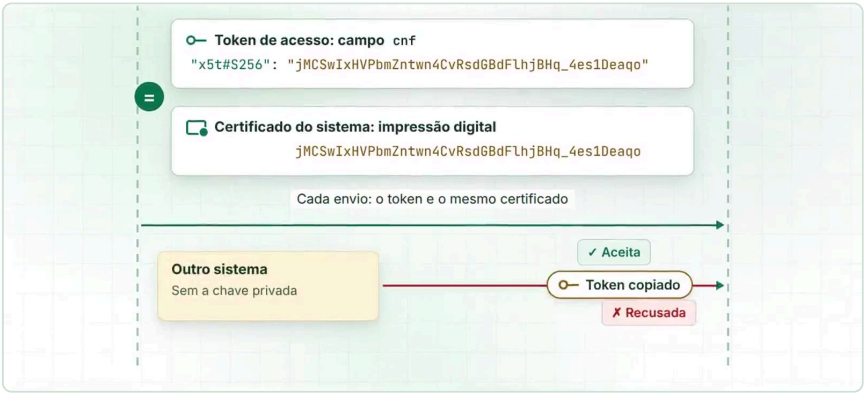


Figura 4.4. O campo `cnf` do token é igual à impressão digital do certificado. Um token copiado, sem a chave privada, é recusado. Quadro do vídeo "Como o sistema obtém o token de acesso".

Renovação e o erro 401

As bibliotecas oficiais guardam o token e pedem um novo automaticamente quando ele está perto de expirar. Ainda assim, uma chamada pode ser recusada com 401. A regra é simples:

Primeiro 401

Descartar o token guardado → Pedir um novo, uma única vez → Repetir a chamada

O 401 se repete

Parar as tentativas → Conferir a credencial

Se o 401 se repete com um token novo, o problema está na credencial, e não na validade do token. Insistir não resolve.

EM RESUMO

- O HubSaúde não usa usuário e senha: o sistema comprova quem é com o certificado digital, e a chave privada nunca sai dele.
- Na conexão mTLS, os dois lados apresentam e conferem certificados.
- O token é obtido em três passos: descoberta, `client_assertion` assinado e troca pelo token.
- O token vale por cinco minutos, traz os escopos liberados e fica vinculado ao certificado pelo campo `cnf`.
- As bibliotecas oficiais fazem todos esses passos e renovam o token.

PARTE 2

Prática guiada

Execute os comandos a partir da pasta `curso-hubsaude`, com o Simulador em funcionamento. Os passos 1 a 5 usam o terminal; o passo 6, um programa em Java.

ATENÇÃO

As variáveis criadas nos passos, como `ASSERTION` e `TOKEN`, existem apenas na janela de terminal em que foram criadas. Execute os passos 1 a 5 na mesma janela.

1 Gerar o client_assertion

Usar o assistente do Simulador para criar e assinar a prova de identidade.

VOCÊ DIGITA

```
$ ASSERTION=$(curl -s --cacert simulador-server.pem -X POST https://localhost:8443/auth/client-assertion \
    -H 'Content-Type: application/json' \
    -d "$(jq -n --arg chave "$(cat client-key.pem)" '{client_id: "meu-sistema", private_key_pem:
  $chave}')" \
    | jq -r .client_assertion)
$ echo "$ASSERTION"
```

O comando guarda o resultado na variável `ASSERTION` e o exibe: um texto longo, em três partes separadas por ponto, que começa com `eyJ`.

ATENÇÃO

Este assistente existe apenas no Simulador e apenas para fins didáticos: ele recebe a chave de teste para montar o bilhete por você. No ambiente oficial, o `client_assertion` é montado pelo seu próprio sistema, normalmente pela biblioteca oficial, e a chave privada nunca é enviada a nenhum servidor.

☐ Passo concluído

2 Ler o conteúdo do client_assertion

Decodificar o cabeçalho e o conteúdo e identificar cada campo.

VOCÊ DIGITA

```
$ echo "$ASSERTION" | jq -R 'split(".")[0] | gsub("-";"+") | gsub("_";"/") | @base64d | fromjson'
$ echo "$ASSERTION" | jq -R 'split(".")[1] | gsub("-";"+") | gsub("_";"/") | @base64d | fromjson'
```

Cada comando separa uma das partes, desfaz a codificação e formata o JSON. O cabeçalho informa o algoritmo de assinatura, `RS384`. O conteúdo traz os campos da seção 4.4:

VOCÊ VÊ

```
{
  "iss": "meu-sistema",
  "sub": "meu-sistema",
  "aud": ["https://localhost:8443/auth/token"],
  "iat": 1793234321,
  "exp": 1793234621,
  "jti": "b39e28bd-17b3-4d44-89f2-b9457bd5e0ad"
}
```

DESAFIO

Calcule `exp` menos `iat`. O resultado, em segundos, é a validade do bilhete: 300 segundos, ou cinco minutos.

☐ Passo concluído

3 Trocar pelo token de acesso

Enviar o client_assertion ao servidor de autorização apresentando o certificado de teste.

VOCÊ DIGITA

```
$ RESPOSTA=$(curl -s --cacert simulador-server.pem --cert client-cert.pem --key client-key.pem \
    -X POST https://localhost:8443/auth/token \
    -d grant_type=client_credentials \
    -d client_id=meu-sistema \
    -d client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer \
    --data-urlencode "client_assertion=$ASSERTION" \
    -d "scope=system/Patient.c system/Patient.rs system/Task.rs")
$ TOKEN=$(echo "$RESPOSTA" | jq -r .access_token)
$ echo "$RESPOSTA" | jq 'del(.access_token)'
```

Parte do comando	O que faz
`--cacert`	Confere o certificado do Simulador: o seu lado da conexão mTLS.
`--cert` e `--key`	Apresentam o certificado de teste: o lado do sistema na conexão mTLS.
`grant_type`	`client_credentials`: comunicação entre sistemas, sem uma pessoa conectada.
`client_assertion`	O bilhete assinado do passo 1, com o tipo informado em `client_assertion_type`.
`scope`	Os escopos pedidos, separados por espaço.

O último comando exibe a resposta sem o token, que é longo:

VOCÊ VÊ

```
{
  "expires_in": 300,
  "scope": "system/Patient.c system/Patient.rs system/Task.rs",
  "token_type": "Bearer"
}
```

OBSERVE
O token fica guardado na variável `TOKEN` e será usado no módulo 5. Como ele vale por cinco minutos, repita os passos 1 e 3 sempre que precisar de um novo.

☐ **Passo concluído**

4 Conferir o vínculo com o certificado

Comparar o campo cnf do token com a impressão digital do certificado de teste.

O primeiro comando lê o campo `cnf` do token; o segundo calcula a impressão digital do certificado de teste no mesmo formato:

VOCÊ DIGITA

```
$ echo "$TOKEN" | jq -R 'split(".")[1] | gsub("-";"+") | gsub("_";"/") | @base64d | fromjson | .cnf'
$ openssl x509 -in client-cert.pem -outform DER | openssl dgst -sha256 -binary | base64 | tr '+/' '-_' |
  tr -d '='
```

VOCÊ VÊ

```
{
  "x5t#S256": "jMCSwIxHVPbmZntwn4CvRsdGBdFlhjBHq_4es1Deaqo"
}
jMCSwIxHVPbmZntwn4CvRsdGBdFlhjBHq_4es1Deaqo
```

OBSERVE
Os dois valores são iguais. No seu computador, o valor será outro, porque o certificado é outro, mas a igualdade se mantém. É ela que o HubSaúde confere em cada chamada: o token só é aceito quando acompanhado do certificado que o originou.

☐ **Passo concluído**

5 Provocar e diagnosticar falhas

Repetir o pedido em situações de erro e reconhecer cada resposta.

Errar de propósito, aqui, ensina a reconhecer as mensagens quando elas aparecerem na integração real. Execute cada situação e compare a resposta:

Situação	Como provocar	Resposta	Por quê
Bilhete reaproveitado	Repita o passo 3 sem gerar um novo `client_assertion`.	`invalid_client`, com a mensagem `jti já utilizado`	Cada bilhete só pode ser usado uma vez.
Pedido sem certificado	Gere um novo bilhete com o passo 1 e repita o passo 3 sem `--cert` e `--key`.	`invalid_client`, com a mensagem `mTLS client certificate required on this endpoint`	Sem o certificado, a conexão não é mTLS.
Escopo não liberado	Gere um novo bilhete e repita o passo 3 com `-d "scope=system/Observation.c"`.	`invalid_scope`	O cadastro não inclui esse escopo.

> **DICA**
> Nas duas últimas situações, gere sempre um bilhete novo antes. Caso contrário, a resposta será a da primeira situação, e a falha que você quer observar fica escondida.

☐ **Passo concluído**

6 Obter o token com a biblioteca oficial

Repetir a obtenção do token a partir de um programa em Java.

Nos passos anteriores, você executou cada etapa à mão. Em um sistema real, a **biblioteca oficial** faz a descoberta, monta e assina o `client_assertion`, troca pelo token e o renova. Este material usa Java; a página do módulo no portal traz o mesmo programa em Python, TypeScript e C#.

Instalar a biblioteca

Enquanto a biblioteca não é publicada no Maven Central, compile-a a partir do código-fonte. O comando `mvn install` a coloca no repositório Maven local do seu computador:

VOCÊ DIGITA

```
$ git clone https://github.com/sesgo-ti/hubsaude-cliente-java.git
$ cd hubsaude-cliente-java
$ mvn install -DskipTests
```

A versão instalada aparece ao final da compilação, como `0.4.1-SNAPSHOT`.

Preparar o projeto

Volte para a pasta `curso-hubsaude`, crie dentro dela a pasta `java` e copie para ela os três arquivos de certificado. Os programas leem esses arquivos da pasta em que são executados.

VOCÊ VÊ

```
curso-hubsaude/java/
├── pom.xml
├── client-key.pem
├── client-cert.pem
```

```
├── simulador-server.pem
└── src/main/java/ObterToken.java
```

Salve o `pom.xml` abaixo na pasta `java`. Ele declara a biblioteca oficial, com a versão que apareceu ao final da compilação (ajuste-a se a sua for outra), e o Jackson, que lê e escreve JSON e será usado no módulo 5. O projeto é compilado para o Java 21, a versão exigida pela biblioteca.

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>br.exemplo</groupId>
    <artifactId>curso-hubsaude</artifactId>
    <version>1.0</version>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>br.gov.go.saude.hubsaude</groupId>
            <artifactId>hubsaude-cliente-java</artifactId>
            <version>0.4.1-SNAPSHOT</version>
        </dependency>
        <dependency>
            <groupId>tools.jackson.core</groupId>
            <artifactId>jackson-databind</artifactId>
            <version>3.2.1</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
            </plugin>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.5.0</version>
            </plugin>
        </plugins>
    </build>
</project>
```

Para baixar: `https://sesgo-ti.github.io/hubsaude/curso/arquivos/java/pom.xml`

Escrever o programa

Salve o programa abaixo como `src/main/java/ObterToken.java`, dentro da pasta `java`:

java/ObterToken.java

```java
import br.gov.go.saude.hubsaude.client.SmartTokenClient;

import java.nio.file.Path;

public class ObterToken {

    public static void main(String[] args) throws Exception {
        try (var client = SmartTokenClient.builder()                          // 1
                .fhirBase("https://localhost:8443")                           // 2
                .clientId("meu-sistema")                                      // 3
                .privateKeyPem(Path.of("client-key.pem"))                     // 4
                .certificatePem(Path.of("client-cert.pem"))                   // 5
                .serverTrustAnchor(Path.of("simulador-server.pem"))           // 6
                .build()) {

            String token = client.obtainToken("system/Patient.c system/Patient.rs system/Task.rs");  // 7
            System.out.println(token.substring(0, 40));
        }
    }
}
```

1. Cria o cliente da biblioteca. O `try` garante que ele seja fechado ao final.
2. O endereço do Simulador. A biblioteca faz a descoberta a partir dele.
3. O `client_id` do cadastro.
4. A chave privada, que assina o `client_assertion`. Ela não sai do computador.
5. O certificado de teste, apresentado na conexão mTLS.
6. O certificado do Simulador, no qual o programa confia.
7. Uma única chamada faz os três passos e devolve o token.

Executar

VOCÊ DIGITA

```
$ mvn -q compile exec:java -Dexec.mainClass=ObterToken
```

O programa exibe os 40 primeiros caracteres do token, que começam com `eyJ`, como todo JWT. O aviso "No SLF4J providers were found", exibido na execução, é inofensivo. Guarde este projeto: ele é a base do envio no módulo 5 e do projeto final.

☐ **Passo concluído**

PARTE 3

Verificação

Responda sem consultar o texto. As respostas estão no final do módulo.

1. Por que o HubSaúde recusa um client_assertion já utilizado?

2. Um token foi obtido com sucesso, mas a primeira chamada com ele foi recusada com 401. Qual é a causa mais provável?

3. O cadastro do sistema libera system/Patient.c. O que acontece se o sistema pedir system/Patient.c system/Observation.c?

4. Uma chamada retornou 401 e, depois de descartar o token guardado e obter um novo, retornou 401 novamente. O que fazer?

5. Qualquer pessoa consegue ler o conteúdo de um client_assertion. Por que isso não é um problema?

Glossário do módulo

client_assertion
Documento assinado com a chave privada que comprova a identidade do sistema no pedido do token.

cnf
Campo do token que guarda a impressão digital do certificado ao qual ele está vinculado.

Descoberta
Consulta ao endereço `/.well-known/smart-configuration`, que informa onde fica o servidor de autorização.

jti
Identificador único de um `client_assertion`, que impede o seu reaproveitamento.

JWT
Formato de texto em três partes, separadas por ponto: cabeçalho, conteúdo e assinatura.

mTLS
Conexão segura em que os dois lados apresentam e conferem certificados.

SMART Backend Services
Padrão internacional, construído sobre o OAuth 2.0, que o HubSaúde segue para autenticar sistemas.

Token de acesso
Autorização temporária, de cinco minutos, que acompanha cada envio.

PRÓXIMO MÓDULO

5. Enviar e acompanhar

Como o seu sistema envia um registro, recebe o comprovante na hora e acompanha o processamento até o resultado.

Respostas da verificação

1. *Por que o HubSaúde recusa um client_assertion já utilizado?*

Porque cada documento tem um identificador único, o `jti`. A recusa impede que alguém que tenha interceptado o documento o reutilize para obter um token.

2. *Um token foi obtido com sucesso, mas a primeira chamada com ele foi recusada com 401. Qual é a causa mais provável?*

A chamada não apresentou o mesmo certificado usado na obtenção do token. Como o token é vinculado ao certificado pelo campo `cnf`, toda chamada precisa usar a mesma conexão com certificado de cliente.

3. *O cadastro do sistema libera system/Patient.c. O que acontece se o sistema pedir system/Patient.c system/Observation.c?*

O pedido é recusado com `invalid_scope`, porque inclui um escopo não liberado. O sistema deve pedir apenas os escopos de que precisa e que foram autorizados.

4. *Uma chamada retornou 401 e, depois de descartar o token guardado e obter um novo, retornou 401 novamente. O que fazer?*

Interromper as tentativas. O problema não está no token, e sim na credencial: o certificado, a chave ou o `client_id` não correspondem ao cadastro.

5. *Qualquer pessoa consegue ler o conteúdo de um client_assertion. Por que isso não é um problema?*

Porque a segurança não está em esconder o conteúdo, e sim na assinatura. Sem a chave privada, não é possível produzir um bilhete com assinatura válida, e um bilhete lido não pode ser reutilizado, por causa do `jti` e da validade curta.